To solve these race conditions, you often need to reach for Thread::Mutex[1] to coordinate memory access between different threads.

Shared memory is especially risky in multi-tenant SaaS environments where a bug could lead to data being accessed by the wrong users in an application.

Concurrency in Ruby becomes very complex as a system scales, so many Ruby developers don't use threads. Instead, it's more common to run everything top-to-bottom in a single thread. Relatively new changes are happening in Ruby that add a new option for true parallelism. We'll look at that new parallelism option later in this section.

Parallelism in Ruby

Different Ruby implementations are available today. The most common, by far, is MRI Ruby. This section will focus on that implementation due to its popularity.

MRI Ruby doesn't have parallelism within a single process. There's a Global Virtual Machine Lock (GVL) that prevents multiple threads from executing in a process at the same time. This is because the Ruby Virtual Machine isn't internally thread-safe, so you really don't want multiple things running in parallel.

The way around this in Ruby has traditionally been to fork multiple Ruby processes. Popular libraries like Puma and Sidekiq take this approach to parallelism (in addition to multi-threading for increased concurrency). Each forked process has its own GVL, so they run fully in parallel with each other. But memory requirements become multiplicative for each process, so machines need to have more RAM to do this in practice.

MRI Ruby is the most popular Ruby implementation. There are other implementations like JRuby and TruffleRuby that are implemented on top of virtual machines that provide parallel execution. There can be compatibility issues and other tradeoffs with these implementations though, which is why their popularity hasn't increased.

Ruby Ractors

Ruby Ractor[2] is a new actor-model abstraction that provides thread-safe concurrency and parallel execution in Ruby. This is very exciting! It's fairly new, so we haven't seen how it will change Ruby, but it has a lot of promise.

1. https://docs.ruby-lang.org/en/3.2/Thread/Mutex.html
2. https://ruby-doc.org/core-3.0.0/Ractor.html